COMP3311 Week 8 Monday Lecture

- Week 08 Monday
- Assignment 2 Database
- Assignment 2 Scripts
- Normalisation
- Relation Decomposition
- Schema (Re)Design
- Boyce-Codd Normal Form
- BCNF Decomposition
- Exercise: BCNF Normalisation (1)
- Exercise: BCNF Normalisation (2)
- Exercise: BCNF Normalisation (3)
- Third Normal Form
- Exercise: 3NF Normalisation (1)
- Exercise: 3NF Normalisation (2)
- Exercise: 3NF Normalisation (3)
- Database Design Methodology
- Relational Algebra
- Projection
- Selection
- Excluding Rel.Operators
- Example Database 1
- Example Database 4.1
- Selection
- Exercise: Rename
- Projection
- Exercise: Projection
- Selection
- Exercise: Selection
- Rename
- Intersection
- Exercise: Union/Intersection
- Difference

◆ Week 08 Monday

In today's lecture ...

- Relational Design Theory: Normalisation
- Relational Algebra: Intro and basic operations

Things to do ...

- Quiz 5 due by 23:59 Friday 3 November
- Assignment 2 due by 23:59 Wednesday 15 November

◆ Assignment 2 Database

Things to note:

- was a pain in the **** to generate
- is a tiny subset of the real UNSW student DB
  - so your favourite program or stream might be missing
  - and most courses have tiny enrolments (< 10)
- some data might not quite make sense (e.g. enrolled in programs, but not in streams)
  - work with the data as supplied, don't second-guess
- some degrees/streams don't have requirements (in the DB)
- focuses on Engineering degrees, but should work on any degree
- some obvious **NOT NULL** constraints were not included
- assumes program/stream requirements have not changed 2019-2023

◆ Assignment 2 Scripts

Write Python/Perl/psql2 scripts to ...

- **q1.py** track proportion of international students over time
- **q2.py** track satisfaction with a given course over time
- **q3.py** print requirements for a stream/program
- **q4.py** produce a transcript, including WAM
- **q5.py** do progression check for current program
- **q6.py** check progression for a proposed program

Will merge Q5 and Q6 ... essentially the same script

```
python3 q5.py z20 ... | tee | python3 q6.py z20 Prog Stream
```

Since output is text, need to be extra careful with output format

◆ Normalisation

Normalisation: branch of relational theory providing design insights.

Makes use of schema normal forms

- to characterise the level of redundancy in a relational schema

And normalisation algorithms which

- provide mechanisms for transforming schemas to remove redundancy

Normalisation draws heavily on the theory of functional dependencies.

◆ Relation Decomposition

The standard transformation technique to remove redundancy:

- decompose relation R into relations S and T

We accomplish decomposition by

- selecting (overlapping) subsets of attributes
- forming new relations based on attribute subsets

Properties: $R = S \cup T$, $S \cap T \neq \emptyset$ and $r(R) = s(S) \bowtie t(T)$

May require several decompositions to achieve acceptable NF.

Normalisation algorithms tell us how to choose S and T.

◆ Schema (Re)Design

Consider the following relation for *BankLoans*:

branchName	branchCity	assets	custName	loanNo	amount
Downtown	Brooklyn	9000000	Jones	L-17	1000
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Downtown	Brooklyn	9000000	Jackson	L-15	1500
Mesa	Horseneck	400000	Jones	L-93	500
Round Hill	Horseneck	8000000	Turner	L-11	900
North Town	Rye	3700000	Hayes	L-16	1300

This schema has all of the *update* anomalies mentioned earlier.

◆ Schema (Re)Design (cont)

To improve the design, decompose the *BankLoans* relation.

The following decomposition is not helpful:

Branch(branchName, branchCity, assets)
CustLoans(custName, loanNo, amount)

because we lose information (about branch in a blue field etc)

Another possible decomposition:

BranchCust(branchName, branchCity, assets, custName)
CustLoans(custName, loanNo, amount)

◆ Schema (Re)Design (cont)

To improve the design, decompose the *BankLoans* relation.

The following decomposition is not helpful:

Branch(branchName, branchCity, assets)
CustLoans(custName, loanNo, amount)

because we lose information (about branch in a blue field etc)

Another possible decomposition:

BranchCust(branchName, branchCity, assets, custName)
CustLoans(custName, loanNo, amount)

◆ Schema (Re)Design (cont)

The *BranchCust* relation instance:

branchName	branchCity	assets	custName
Downtown	Brooklyn	9000000	Jones
Redwood	Palo Alto	2100000	Smith
Perryridge	Horseneck	1700000	Hayes
Downtown	Brooklyn	9000000	Jackson
Mesa	Horseneck	400000	Jones
Round Hill	Horseneck	8000000	Turner
North Town	Rye	3700000	Hayes

◆ Schema (Re)Design (cont)

The *CustLoans* relation instance:

custName	loanNo	amount
Jones	L-17	1000
Smith	L-23	2000
Hayes	L-15	1500
Jackson	L-15	1500
Jones	L-93	500
Turner	L-11	900
Hayes	L-16	1300

◆ Schema (Re)Design (cont)

Now consider the result of *BranchCust Join CustLoan*:

branchName	branchCity	assets	custName	loanNo	amount
Downtown	Brooklyn	9000000	Jones	L-17	1000
Downtown	Brooklyn	9000000	Jones	L-93	500
Redwood	Palo Alto	2100000	Smith	L-23	2000
Perryridge	Horseneck	1700000	Hayes	L-15	1500
Perryridge	Horseneck	1700000	Hayes	L-16	1300
Downtown	Brooklyn	9000000	Jackson	L-15	1500
Mesa	Horseneck	400000	Jones	L-93	500
Mesa	Horseneck	400000	Jones	L-17	1000
Round Hill	Horseneck	8000000	Turner	L-11	900
North Town	Rye	3700000	Hayes	L-16	1300
North Town	Rye	3700000	Hayes	L-15	1500

◆ Schema (Re)Design (cont)

This is clearly not a successful decomposition.

The fact that we ended up with extra tuples is symptomatic of losing some critical "connection" information during the decomposition.

Such a decomposition is called a lossy decomposition.

In a good decomposition, we should be able to reconstruct the original relation exactly:

If R is decomposed into S and T, then $S \bowtie T = R$

Such a decomposition is called lossless join decomposition.

◆ Boyce-Codd Normal Form

A relation schema R is in BCNF w.r.t set F of functional dependencies iff:

for all $\alpha\beta, X \rightarrow Y$ in F^+

- either $X \rightarrow Y$ is trivial (i.e. $Y \subseteq X$)
- or X is a superkey (i.e. over entire superset of attributes in key)

A DB schema is in BCNF if all of its relation schemas are in BCNF.

Observations:

- any two-attribute relation is in BCNF
- any relation with key K other attributes Y, and $K \rightarrow Y$ is in BCNF

◆ Boyce-Codd Normal Form (cont)

If we transform a schema into BCNF, we are guaranteed:

- no update anomalies due to KF-based redundancy
- lossless join decomposition

However, we are not guaranteed:

- the new schema preserves all fds from the original schema

If we need to preserve dependencies, use 3NF.

A dependency $X \rightarrow Y$ is preserved if all of its attributes exist in one table.

Decomposition may e.g. place X and Y in separate tables

◆ BCNF Decomposition

The following algorithm converts an arbitrary schema to BCNF:

```
Inputs: schema R, set F of fds
Outputs: set Res of BCNF schemas (tables)

Res = {R}
while (any schema S ∈ Res is not in BCNF) {
    choose any fd X → Y in S that violates BCNF
    Res = (Res-S) ∪ (S-Y) ∪ XY
}
```

The last step means: make a table from XY, drop Y from table S, drop table S

The "choose any" step means that the algorithm is non-deterministic

◆ Exercise: BCNF Normalisation (1)

Recall the *BankLoans* schema:

BankLoans(branchName, branchCity, assets, custName, loanNo, amount)

Has functional dependencies F

- *branchName* → *assets,branchCity*
- *loanNo* → *amount,branchName*

The key for *BankLoans* is *branchName,custName,loanNo*

Use the BCNF decomposition algorithm to produce a BCNF schema.

◆ Exercise: BCNF Normalisation (2)

Recall the following table (based on e.g. a spreadsheet):

#F	**Name**	**Address**	**Motes**	**#F**	**Name**	**Address**
3906	Gilfoy Jill-Jill	31 High St.	bachelors lush	4004	Bob	4AB71A
3911	Ng-Yen Lilian	47 High St.	A11 UK	3044	Bob	4AB71A
3911	Ng-Yen Lilian	47 High St.	Bmw Jaguar	3911	Lilian	47 High St.
3911	Ng-Yen Lilian	47 High St.	Bmw Jaguar window	3044	Bob	4AB71A
3911	Ng-Yen Lilian	47 High St.	A11 UK	3911	Lilian	47 High St.
3911	Ng-Yen Lilian	47 High St.	A11 UK	3042	Peter	2FD123

Recall the functional dependencies identified previously,

Use these to convert the table to a BCNF schema.

◆ Exercise: BCNF Normalisation (3)

Consider the schema R and set of fds F

$R = ABCDEFGH$

$F = ABH \rightarrow C, A \rightarrow DE, BGH \rightarrow F, F \rightarrow ADH, BH \rightarrow GE$ }

Produce a BCNF decomposition of R.

◆ Third Normal Form

A relation schema R is in 3NF w.r.t a set F of functional dependencies iff:

for all $\alpha\beta, X \rightarrow Y$ in F^+

- either $X \rightarrow Y$ is trivial (i.e. $Y \subseteq X$)
- or X is a superkey
- or Y is a single attribute from a key

A DB schema is in 3NF if all relation schemas are in 3NF.

The extra condition represents a slight weakening of BCNF requirements.

◆ Third Normal Form (cont)

If we transform a schema into 3NF, we are guaranteed:

- lossless join decomposition
- the new schema preserves all of the fds from the original schema

However, we are not guaranteed:

- no update anomalies due to KF-based redundancy

Whether to use BCNF or 3NF depends on overall design considerations.

◆ Third Normal Form (cont)

The following algorithm converts an arbitrary schema to 3NF:

```
Inputs: schema R, set F of fds
Outputs: set R0 of 3NF schemas

  let F0 be a minimal cover for F
  Res = {}
```

```
for each tuple t1 in relation r
  for each tuple t2 in relation r
    if t1[X] = t2[X]
      if t1[Y] ≠ t2[Y]
        violation found
```

- Tuples t_1 and t_2 violate the FD if they agree on X but disagree on Y.
- Checking requires comparing every pair of tuples — $O(n^2)$ in the worst case.

◆ Third Normal Form (cont)

Critical step is producing *minimal cover* F_c for F

A set F of FDs is *minimal* if

- every RHS of $X \rightarrow Y$ is *simple*
  - *(Y is a single attribute)*
- every RHS of $X \rightarrow Y$ is *left-reduced*
  - *(no $Z \subset X$ such that $Z \rightarrow Y$ is also implied by F)*
- every RHS of $X \rightarrow Y$ is *necessary*
  - *(no $X \rightarrow Y$ can be removed without changing F^+)*

Algorithm: right-reduce, left-reduce, eliminate redundant fds

◆ Exercise: BCNF Normalisation (1)

Recall the *BankLoans* schema:

BankLoans(branchName, branchCity, assets, custName, loanNo, amount)

Has functional dependencies F

- *branchName → assets,branchCity*
- *loanNo → amount,branchName*

The key for *BankLoans* is *branchName,custName,loanNo*

Use the 3NF decomposition algorithm to produce a 3NF schema.

◆ Exercise: 3NF Normalisation (2)

Recall the following table (based on e.g. a spreadsheet):

PF	Wines	Address	Wrtes	BF	Name	Cuvfmy
004	21/06 15:15	55 High St.	Parktown Inah	0004	Bob	ABF754
001	04/06 11:10	67 High St.	All UR	0004	Bob	ABF754
004	02/07 12:20	55 High St.	All UR	0003	Dave	XFG123
003	01/07 14:05	12 Low St.	Ashley Wine	0001	Alice	ABC123
001	03/07 15:00	67 High St.	Cooking Inp	0004	Bob	ABF754
002	13/07 12:00	12 High St.	All UR	0002	Peter	XFG123

- 5 rows

Recall the functional dependencies identified previously.

Use these to convert the table to a 3NF schema.

◆ Exercise: 3NF Normalisation (3)

Consider the schema R and set of FDs F

$R = ABCDEFGH$

$F = F_1 = \{BH \rightarrow C, A \rightarrow D, C \rightarrow E, F \rightarrow A, E \rightarrow H, BGH \rightarrow E\}$

Produce a 3NF decomposition of R.

◆ Database Design Methodology

To achieve a "good" database design:

- identify attributes, entities, relationships → ER design
- map ER design to relational schema
- identify constraints *(including keys and functional dependencies)*
- apply BCNF/3NF algorithms to produce normalised schema

Note: *may subsequently need to "denormalise" if the design yields inadequate performance.*

◆ Relational Algebra

Relational algebra (RA) can be viewed as ...

- mathematical system for manipulating relations, or
- data manipulation language (DML) for the relational model

Relational algebra consists of

- operands relations, or variables representing relations
- operators that map relations to relations
- rules for combining operands/operators into expressions
- rules for evaluating such expressions

Why is it important?

- because it forms the basis for DBMS implementation
- relational algebra ops are like machine code for DBMSs

◆ Relational Algebra (cont)

Core relational algebra operations:

- selection: choosing a subset of tuples/rows
- projection: choosing a subset of attributes/columns
- product, join: combining relations
- union, intersection, difference: combining relations
- rename: change names of relations/attributes

Common extensions include:

- aggregation, projection++, division

◆ Relational Algebra (cont)

Select, project, join provide a powerful set of operations for building relations and extracting interesting data from them.

Adding set operations and renaming makes RA complete.

◆ Notation

Standard treatments of relational algebra use Greek symbols.

We use the following notation (because it is easier to reproduce):

Operation	Standard Notation	Our Notation
Selection	$\sigma_{condition}(Rel)$	*Select(Rel)*
Projection	$\pi_{A,B,C}(Rel)$	*Proj(A,B,C,Rel)*
Join	$Rel_1 \bowtie_{Rel_1.Rel_2} Rel_2$	*Join(Rel1, Rel2)*
Rename	$\rho_{a/a1,b/b1}Rel$	*Rename(a/a1,b/b1,Rel)*

For other operations (e.g. set operations) we adopt the standard notation.
Except when typing in a text file, where + = intersection, - = union

◆ Describing RA Operations

We define the semantics of RA operations using

- "conditional set" expressions *e.g. { X | condition on X }*
- tuple notations:
  - *t[A,B]* (extracts attributes A and B from tuple *t*)
  - *t || t2* (concatenated tuples: specify attribute values)
- quantifiers, set operations, boolean operators

For each operation, we also describe it operationally:

- give an algorithm to compute the result, tuple-by-tuple

◆ Describing RA Operations (cont)

All RA operators return a result of type relation.

For convenience, we can name a result and use it later.

E.g.

```
Temp = R op1 S op2 T
Res  = Temp op3 U
```

- *Temp* is a temporary variable
- *Res* is the result of the query

Each result is a relation with a well-defined schema

- this applies equally to intermediate results and final results

◆ Example Database #1

R

A	B	C	D
a	1	x	m
b	2	y	n
c	3	z	o
d	4	x	p
e	5	y	q
f	2	x	r

S

D	E	F
1	a	x
3	b	y
5	c	x
7	d	y
9	e	x
5	b	x

◆ Example Database #2

```
Students(name,email)
  (JD, Carters)
  (Alice, Alice)
  (Ray, Crayons)
  (Fiona, Mallinsons)

Employees(name,addr,licence)
  (CBH, Coogee, 4001D)
  (Alice, Alice St, 1111D)
  (Rogers Kingsford, 1101D)
  (Fiona, Bondi, 1001D)

Sailing(boat,date,crew)
  (CBH, Mar, 2.30)
  (CBH, Mar, 4.30)
  (Royal, Porter, 3.00)
  (Regent, Nov, 1.30)

Likes(drinker,beer)
  (Alice, Tooth)
  (Andrew, Tooth)
  (Daniel, Porter)
  (John, XB)

Drinkers(name,addr,phone)
  (John, Alexandria, 9311108)
  (Andrew, Bondi Junction, 9387766)
  (Andrew, Clarke, 9417100)

Frequents(drinker,bar)
  (John, Regent, Coogee Bay)
  (John, Regent)
  (John, Royal)
  (Andrew, Royal)
  (Andrew, Royal)
```


◆ Rename

Rename provides "schema mapping".

If expression E returns a relation $R(A_1, A_2 ... A_n)$ then

$Rename(S(B_1, B_2 ... B_n)E)$

gives a relation called S

- containing the same set of tuples as E
- but with the name of each attribute changed from A_i to B_i

Rename is like the identity function on the contents of a relation

The only thing that Rename changes is the schema.

◆ Rename (cont)

Rename can be viewed as a "technical" apparatus of RA.

Can use implicit rename/project in sequences of RA operations, e.g.

```
Res = Rename(Rest(t,r),Proj(Rest(t,r),(Del(a=5)(R)) Join R)

Tmp1 = Del(a=5)(R)
Tmp2 = Proj(Rest(t,r)(Tmp1)
Tmp3 = Rest(t,r)(Tmp2) Join R(t)
Tmp4 = Tmp2 Join(cond) Tmp3
Res  = Rename(Rest(t,r),R)(Tmp4)

Tmp1(a,b,c) = Del(a=5)(R)
Tmp2(cat,d) = Rest(t,r)(Tmp1)
Res(a,b,d)  = Tmp2 Join(cond) Tmp3
```

In SQL, can achieve a similar effect by defining a set of views

◆ Exercise: Rename

Answer the following in SQL then relational algebra

- rename the columns in the Beers relation (beer,brewer)
- rename the **addr** column in Drinkers in **suburb**
- change the name of the Bars relation to Pubs

◆ Projection

Projection returns a set of tuples containing a subset of the attributes in the original relation.

$\Pi_R(r) = Proj(R)(r) = \{ t[R] \mid t \in r \}$, where $t[R]$

t specifies a subset of the attributes of *R*

Note that removing key attributes can produce duplicates.

In RA, duplicates are removed from the result set.

Result size: $|\Pi_R(r)| \leq |r|$ Result schema: *R'[X]*

Algorithmic view:

```
result = {}
for each tuple t in relation r
  result = result U {t[X]}
```


◆ Projection (cont)

Examples of projection:

R

A	B	C	D
1	2	x	y
5	2	x	y
1	2	x	z
4	2	x	y
4	2	x	z

Proj(A,B,C)(R)

A	B	C
1	2	x
5	2	x
4	2	x

Proj(B,D)(R)

B	D
2	y
2	z

Proj(D)(R)

D
y
z

◆ Exercise: Projection

Answer the following in SQL then relational algebra

- Names of all beers
- What are all of the breweries?
- Names of drinkers who live in Newtown

◆ Selection

Selection returns a subset of the tuples in a relation *r(R)* that satisfy a specified condition *C*.

$\sigma_C(r) = Sel(C)(r) = \{ t \mid t \in r \wedge C(t) \}$

C is a boolean expression on attributes in *R*

Result size: $|\sigma_C(r)| \leq |r|$

Result schema: same as the schema of *r* (i.e. *R*)

Algorithmic view:

```
result = {}
for each tuple t in relation r
  if C(t) {} {result = result U {t} }
```


◆ Selection (cont)

Examples of selection:

R

A	B	C	D
a	1	x	4
b	2	y	5
c	4	z	4
d	8	x	5
e	1	y	4
f	2	x	5

Sel[B=1](R)

A	B	C	D
a	1	x	4
e	1	y	4

Sel[B>=D](R)

A	B	C	D
c	4	z	4
d	8	x	5

Sel[A=C](R)

A	B	C	D

Sel[A=b or A=c](R)

A	B	C	D
b	2	y	5
c	4	z	4

❖ Exercise: Selection

Answer the following in SQL and relational algebra

- details of all bars in The Rocks
- beers made by Sierra Nevada
- beers sold in the Coogee Bay Hotel

❖ Union

Union combines two compatible relations into a single relation via set union of sets of tuples.

$r_1 \cup r_2 = \{ t \mid t \in r_1 \vee t \in r_2 \}$, where $r_1(R), r_2(R)$

Compatibility = both relations have the same schema

Result size: $|r_1 \cup r_2| \leq |r_1| + |r_2|$ Result schema: R

Algorithmic view:

```
result = r1
for each tuple t in relation r2
    result = result ∪ {t}
```


❖ Intersection

Intersection combines two compatible relations into a single relation via set intersection of sets of tuples.

$r_1 \cap r_2 = \{ t \mid t \in r_1 \wedge t \in r_2 \}$, where $r_1(R), r_2(R)$

Uses same notion of relation compatibility as union.

Result size: $|r_1 \cup r_2| \leq \min(|r_1|,|r_2|)$ Result schema: R

Algorithmic view:

```
result = {}
for each tuple t in relation r1
    if (t ∈ r2) { result = result ∪ {t} }
```


❖ Intersection (cont)

Examples of union and intersection:

T = Sel[B=1](R)

A	B	C	D
a	1	x	4
e	1	y	4

U = Sel[C=x](R)

A	B	C	D
a	1	x	4
d	8	x	5

T union U

A	B	C	D
a	1	x	4
d	8	x	5
e	1	y	4

T intersect U

A	B	C	D
a	1	x	4

❖ Exercise: Union/Intersection

Answer the following in SQL then relational algebra

- Bars where either John or Gernot drinks
- Bars where both John and Gernot drink

❖ Difference

Difference finds the set of tuples that exist in one relation but do not occur in a second compatible relation.

$r_1 - r_2 = \{ t \mid t \in r_1 \wedge t \notin r_2 \}$, where $r_1(R), r_2(R)$

Uses same notion of relation compatibility as union.

Note: tuples in r_2 but not r_1 do not appear in the result

- i.e. set difference != complement of set intersection

Algorithmic view:

```
result = {}
for each tuple t in relation r1
    if (!(t ∈ r2)) { result = result ∪ {t} }
```


❖ Difference (cont)

Examples of difference:

T = Sel[B=1](R)

A	B	C	D
a	1	x	4
e	1	y	4

U = Sel[C=x](R)

A	B	C	D
a	1	x	4
d	8	x	5

T - U

A	B	C	D
e	1	y	4

U - T

A	B	C	D
d	8	x	5

Clearly, difference is not symmetric.

❖ Difference (cont)

Answer the following in SQL then relational algebra

- Bars where John drinks and Gernot doesn't
- Bars that sell VB but not New